Contents:

1. Task 1: Generated an OHLCComputer Class ..... Page 2

2. Task 1: Output in terminal…… Page 3

3. Task 2: CandlestickPlotter Class and Plotting ..... Page 4

4. Task 2: Output in terminal…… Page 5

5. Task 3: Country-wise Data Filter ..... Page 6

6. Task 3: Output in terminal…… Page 6

7. Task 4: Predicting Future Temperature Trends ..... Page 7

8. Task 4: Output in terminal…… Page 7

9. Originality and Challenges ..... Page 8

10. Class Diagram ..... Page 9

OBJECT-ORIENTED-PROGRAMMING

TASK 1:

Generated an OHLCComputer Class:
I made an OHLCComputer class for this task so that it could compute the Open, High, Low, and Close (OHLC) values for temperature data based on the resolution (daily, monthly, or yearly). The class includes a method named calculateOHLC which takes the input as the temperature data, the country code, the time resolution, and the start and end dates. This method takes the data and turns it into a vector of OHLC candlestick data that shows the overall temperature trends for the given parameters.

Inside the OHLCComputer Class:
The calculateOHLC Method:
The country code, the number, the date range, and the temperature data (in the form of a 2D vector) are all things that this method needs. It starts by matching the column name style <country_code>_temperature to find the correct column in the dataset for the given country. After finding the column, the data is grouped based on the resolution (daily, monthly, or yearly). This is done by using helper methods to get the right date or time frame from the timestamp. When the data is for a year, like 1980, the group key is the year. When the data is for a month, it's the year and the month, like 1980-01.

After that, the method filters the data so that it only includes rows that happen between the dates you gave it. Temperatures are kept in a system with groups, and each group represents a different time period, like a year. The method computes the High that is the maximum temperature, Low is the minimum temperature, and Close is the average temperature for this group. If there is no prior data, the Open is calculated by taking the average temperature of the last group. If there is no prior data, the Close is used instead. Standard library methods like std::max_element, std::min_element, and std::accumulate are used to figure out these numbers.

The OHLC values for a group are calculated and stored in a vector. This process is repeated for all groups in the date range. Each OHLC data entry is shown as a row in a 2D vector, which is what the method returns at the end.

The Data Representation and Constructor:
This method calculates the OHLC values (Open, High, Low, and Close) and the time frame that goes with them (for example, a year). The results are stored in the returned data structure. These numbers are displayed as data from the program's output, so users can see how the temperatures are changing over time and in the country they chose. The method makes sure that the data is processed dynamically based on the resolution and date range that the user chooses. This makes the data flexible and useful for a variety of analyses.

How users interact:
There is a simple menu interface that lets users deal with the system. They select a country from a list, select a time resolution, and choose a set of dates to look at. The program then computes and displays the OHLC candlestick data that goes with it, which makes it easy for users to see and understand temperature trends for the parameters they picked. Users can easily do weather analyses and see the results because of the interface that is intended to be simple and effective.
TASK 1 OUTPUT DISPLAYED ON THE TERMINAL:

```
========== WEATHER ANALYSIS DATA IMPLEMENTATIONS ==========
1. View the filtered OHLC table and Candlestick plot
2. Prediction (Task 4)
0. Exit
====================================
Select an option: 1

========== Countries To Choose From==========
1. AT
2. BE
3. BG
4. CH
5. CZ
6. DE
7. DK
8. EE
9. ES
10. FI
11. FR
12. GB
13. GR
14. HR
15. HU
16. IE
17. IT
18. LT
19. LU
20. LV
21. NL
22. NO
23. PL
24. PT
25. RO
26. SE
27. SI
28. SK
0. Back
======================================
Select a country by number (or 0 to go back): 1

You selected: AT

========== Select the Time Resolution ==========
1. Daily
2. Monthly
3. Yearly
0. Go Back
======================================
Select a resolution by number (or 0 to go back): 3
Enter the start date (YYYY-MM-DD): 1980
Enter the end date (YYYY-MM-DD): 2019
```

OHLC Data (yearly) for AT:

```
================= OHLC Table for plotting =================
        Date        Open        High         Low       Close
        1980    6.049147   29.132000  -14.507000    6.049147
        1981    6.049147   29.419000  -16.219000    7.191993
        1982    7.191993   28.395000  -15.084000    7.566535
        1983    7.566535   32.416000  -18.098000    8.053653
        1984    8.053653   32.659000  -13.338000    6.835889
        1985    6.835889   29.166000  -22.705000    6.576975
        1986    6.576975   29.785000  -20.601000    7.120910
        1987    7.120910   28.054000  -25.301000    6.599801
        1988    6.599801   31.313000  -13.019000    7.688502
        1989    7.688502   28.968000  -10.969000    8.067289
        1990    8.067289   29.145000  -11.347000    8.067452
        1991    8.067452   30.448000  -15.978000    6.926014
        1992    6.926014   32.326000  -13.679000    8.042797
        1993    8.042797   30.092000  -17.096000    7.390172
        1994    7.390172   31.458000  -12.935000    8.698222
        1995    8.698222   31.620000  -14.468000    7.331365
        1996    7.331365   27.225000  -20.283000    5.976157
        1997    5.976157   27.528000  -12.095000    7.322460
        1998    7.322460   31.945000  -15.665000    7.846743
        1999    7.846743   29.667000  -16.172000    7.760209
        2000    7.760209   32.539000  -16.425000    8.763799
        2001    8.763799   30.489000  -17.771000    7.675269
        2002    7.675269   30.305000  -16.596000    8.420704
        2003    8.420704   33.745000  -17.103000    8.113893
        2004    8.113893   28.997000  -15.015000    7.290176
        2005    7.290176   31.314000  -19.122000    6.978591
        2006    6.978591   30.550000  -21.521000    7.615344
        2007    7.615344   32.701000  -10.583000    8.545848
        2008    8.545848   28.851000  -11.306000    8.357880
        2009    8.357880   29.917000  -15.502000    7.897676
        2010    7.897676   30.419000  -18.188000    6.762714
        2011    6.762714   32.679000  -13.038000    8.456161
        2012    8.456161   32.540000  -17.832000    8.201517
        2013    8.201517   32.556000  -15.969000    7.677337
        2014    7.677337   29.589000   -9.640000    9.065739
        2015    9.065739   32.577000  -11.174000    9.057319
        2016    9.057319   29.467000  -12.179000    8.392350
        2017    8.392350   32.631000  -14.708000    8.331979
        2018    8.331979   30.307000  -17.596000    9.213405
============================================
```

TASK 2:

This was a task where I needed my teachers' assistance as it was a challenge to create and plot candlesticks.

CandlestickPlotter Class and plotCandlesticks Method

The candlestick graph in the code is made with the CandlestickPlotter class, specifically within the plotCandlesticks method. This method takes OHLC data (Open, High, Low, and Close figures for temperature trends over a certain time frame), it gives you a text-based visualisation. The graph uses characters like | to display the stalk (the range of highs and lows) and - to show the Open and Close numbers. In addition, colours (red and green) show whether the temperature increased or decreased during that time period, which makes the graph easy to understand visually.

Normalizing Temperature Values for Plotting

Before making the graph, the method uses the normalise function to make the temperature figures fit within a plot height that has already been set. The lowest and highest temperatures in the data are used to scale each temperature number and match it to the available height in rows. To keep the graph from looking too crowded, a small space is added above and below the highest and lowest temperature numbers. Depending on the precision of the OHLC data, the horizontal axis (x-axis) shows time frames like days, months, or years. The vertical axis (y-axis) shows the normalised temperature values.

Building the Candlestick Graph

The graph is made by going from the highest temperature to the lowest temperature, row by row. The stalk of each candlestick is a vertical line (|), which shows the spread of values between the High and Low points. The Open and Close numbers are shown by horizontal bars (-) in the right places. How Open and Close relate to each other determines the colour of the bars: green means the temperature went up (Open < Close), while red means it went down (Open > Close). The labels for the time frames are properly spaced along the x-axis, which makes the graph clear even for bigger datasets.

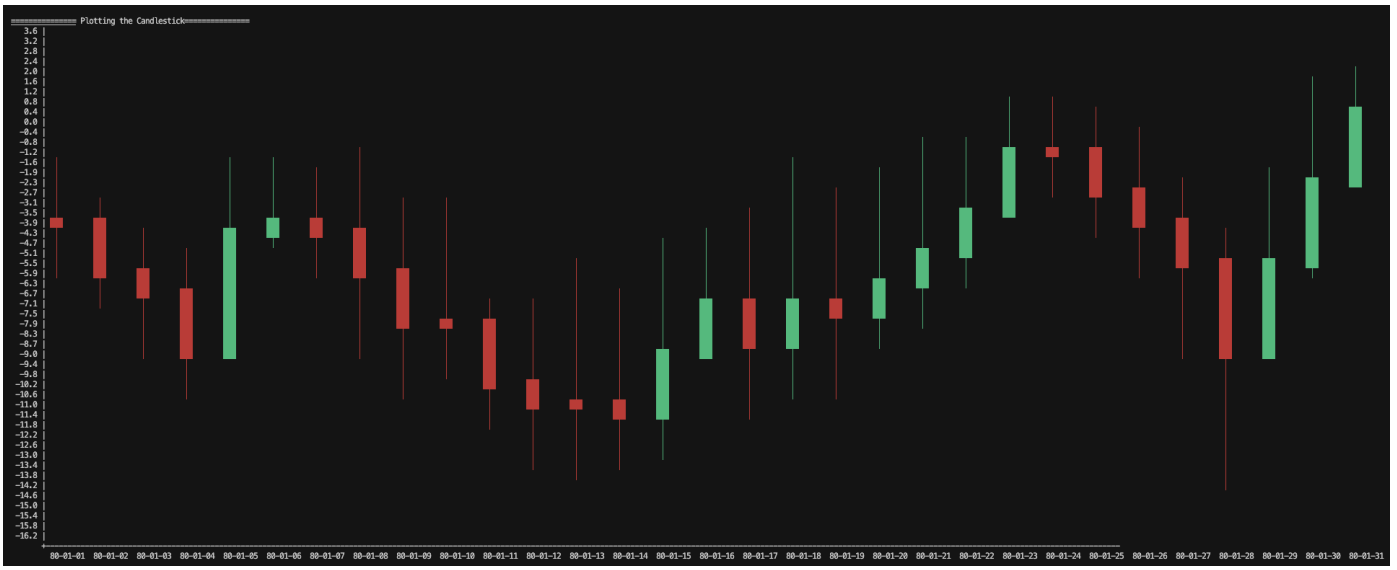
TASK 2 OUTPUT DISPLAYED ON TERMINAL:

```
========== WEATHER ANALYSIS DATA IMPLEMENTATIONS ==========
1. View the filtered OHLC table and Candlestick plot
2. Prediction (Task 4)
0. Exit
====================================
Select an option: 1

========== Countries To Choose From==========
1. AT
2. BE
3. BG
4. CH
5. CZ
6. DE
7. DK
8. EE
9. ES
10. FI
11. FR
12. GB
13. GR
14. HR
15. HU
16. IE
17. IT
18. LT
19. LU
20. LV
21. NL
22. NO
23. PL
24. PT
25. RO
26. SE
27. SI
28. SK
0. Back
========================================
Select a country by number (or 0 to go back): 1

You selected: AT

========== Select the Time Resolution ==========
1. Daily
2. Monthly
3. Yearly
0. Go Back
========================================
Select a resolution by number (or 0 to go back): 1
Enter the start date (YYYY-MM-DD): 1980-01-00
Enter the end date (YYYY-MM-DD): 1980-01-31
```

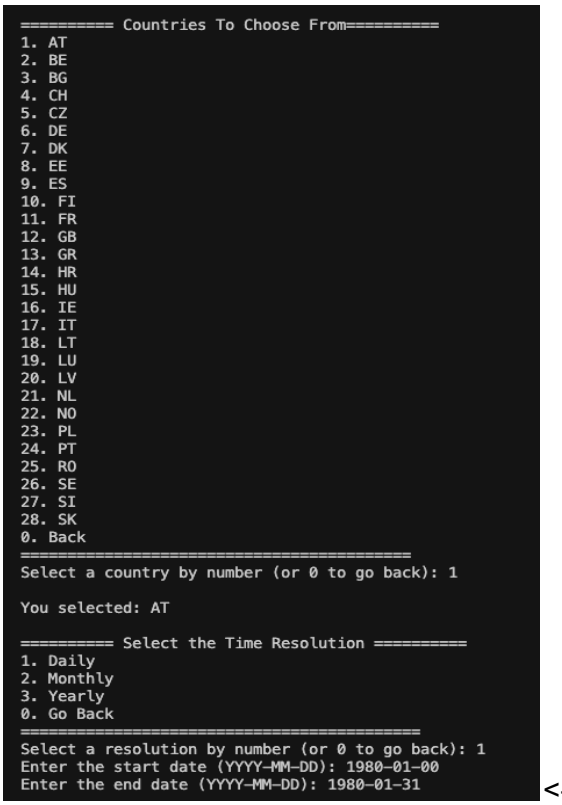


TASK 3

Country-wise Data Filter:
The first filter allows users to select a country from a menu that is created dynamically. To make this menu, the dataset headers are parsed to get the country codes, such as GB for the UK or FR for France. Once the user chooses a country, only the temperature data for that country is used for the rest of the analysis. This is done by finding the correct column in the dataset, which has the style _temperature. The data for the chosen country is then passed to the OHLCComputer class so that the OHLC candlestick data can be generated.

Time Resolution and Date Range Filter:
The second filter allows users to decide the time range for the resolution (daily, monthly, or yearly). Once users have chosen the country, they can choose when the analysis starts and finishes. These inputs tell the OHLCComputer method how to group the temperature data based on the scale the user selected and filter it so that it only shows rows from the given date range. If the user chooses "yearly" resolution and a date range of 1980 to 1990, for example, the OHLC data is calculated and grouped by year, and only data from this range is handled.

Filtering and Plotting the Text-Based Candlestick Graph:
The OHLC data is sent to the CandlestickPlotter class after it has been filtered based on the country, resolution, and date range that was chosen. The plotCandlesticks method in this class makes a text-based candlestick graph. The x-axis shows the chosen time frames, and the y-axis shows the normalised temperature values. Each candlestick shows a different amount of time, for example a year. The stalk (|) shows the spread of High and Low values, and the horizontal bars (-) show the Open and Close values. The line is colour-coded, with green showing that the temperature is rising (Open < Close) and red showing that it is falling (Open > Close). The date range makes sure that the graph only shows the important data.



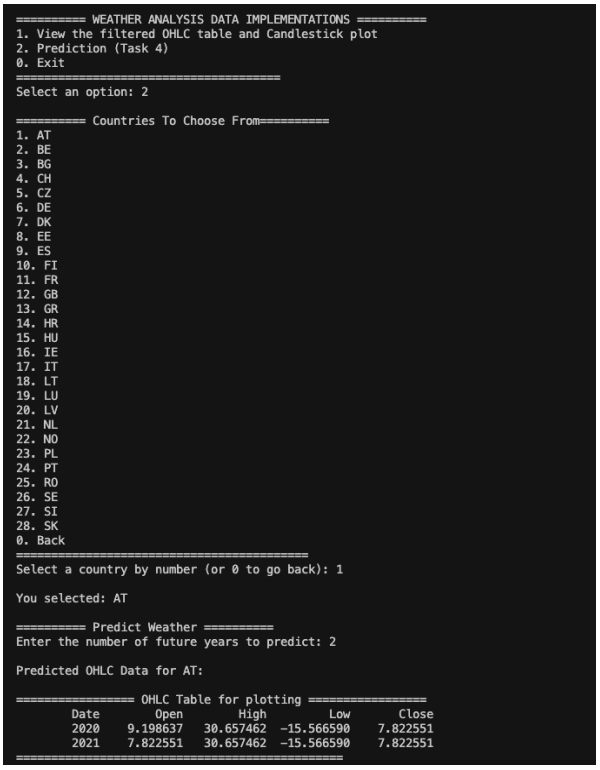
<-- TASK 3 OUTPUT ON THE TERMINAL:

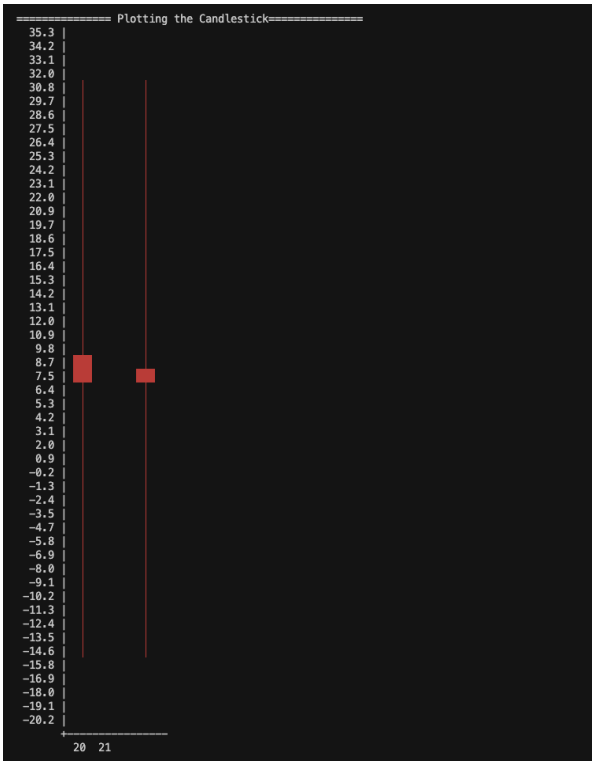
TASK 4

Task 4 of my code is designed to predict future temperature trends of a certain country by looking at historical data from 1980 to 2019. The task is implemented as an option in the main menu, where the user selects a country from a dynamically generated list. After selecting a country, the program uses the mean averages of past OHLC data to predict the Open, High, Low, and Close (OHLC) temperature values for 2020 and other years. The predictions are then put on a text-based candlestick graph so that you can see how things will change in the future.

Prediction Model:
Using the averages of past OHLC data, the predictTemperature method in the PredictionSystem class figures out what the values will be in the future. Each year's Open is set to the previous year's Close. The High, Low, and Close are based on the averages of past values. This method does a good job of capturing broad long-term trends.

Text-Based Plotting:
The expected data is sent to the CandlestickPlotter class, which uses text characters to make a graph. The y-axis shows the temperatures, and the x-axis shows the years that are expected to happen. Candlesticks use | for High-Low ranges and - for Open-Close ranges. When trends are rising, bars are green, and when trends are falling, bars are red.
TASK 4 OUTPUT ON THE TERMINAL:





ORIGINALITY AND CHALLENGES

ORIGINALITY:

1. Main Menu for Candlestick Plotting and Prediction:
The main menu at the start of the program provides clear options for users to either plot candlestick graphs or predict future temperature trends. This separation of functionalities into different tasks makes the user experience better by letting them focus on specific analyses. The interface is easy to use and responds quickly to changes made by the user, like selecting a country or a set of dates.

2. Country Specific Filters and Dynamic Interaction:
The implementation allows users to interact actively with the dataset by including filters for data based on country, time resolutions (daily, monthly, yearly), and date ranges. Users can pick a country from a list, select the resolution they want, and select a time range as well. This makes the analysis specific and useful and makes sure that the program can meet the needs of a wide range of users.

3. Prediction for Multiple Years:
Task 4's prediction feature figures out how temperatures will change in the coming years. This means making predictions not only for 2020 but also for years after that based on the average OHLC readings of past data. The Open value is dynamically linked to the Close value from the previous year to keep things making sense. This feature gives the predictive model more depth and lets you look at long-term trends.

4. ANSI for Colour Coding: One interesting and cool thing about candlestick plots is that they can be coloured using ANSI escape codes. When temperatures go up (Open < Close), the bars are green. When temperatures go down, the bars are red (Open > Close). This use of different colours makes the text-based graph easier to read and gives user a quick visual clue for trends.

CHALLENGES:

1. Dynamic Filtering and Grouping of Data: It was hard to dynamically filter and group temperature data based on resolutions and date ranges to be chosen by the user. The code handles big datasets quickly by grouping data using helper functions like createGroupKey and isWithinRange. This makes sure that the OHLC values are added up correctly. It took careful application of filtering logic to process years of data while keeping performance and accuracy high.

2. Text-Based Candlestick Graph and Spacing:
It was hard to get the spacing and orientation right when I was making a text-based candlestick graph. It took careful normalisation and plotting logic to make sure that the candlesticks were properly spaced along the x-axis and that the High, Low, Open, and Close values were shown correctly. Using symbols like | for stalks and - for bars, along with the right amount of space between them, made sure that the graph was accurate and easy to understand.

3. Model for Predicting Temperature Trends:
Adding a model for predicting future temperature trends made things more complicated. Based on past data, the model figures out averages for High, Low, and Close. It also uses the previous year's Close to make sure that Open numbers change smoothly. This logical consistency makes predictions more likely to come true. Balancing the simplicity of the model with meaningful results required thoughtful design and implementation.

4. User-Friendly Menu System:
Putting all of these features together in a menu-driven system took a lot of consideration about how to make it easy for people to use and interact with, especially the main menu. Dynamically creating choices for choosing a country, entering a date range, and choosing a resolution made sure that the user experience was smooth. Putting all of these features together into a strong and cohesive system shows how carefully the code was planned.

CLASS DIAGRAM

